患者模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 患者实体表(patient)

```sql
CREATE TABLE `patient` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '患者ID',
  `user_id` bigint NULL DEFAULT NULL COMMENT '用户ID',
  `patient_no` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT
NULL COMMENT '患者编号',
  `name` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT NULL
COMMENT '姓名',
  `id_card` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
DEFAULT NULL COMMENT '身份证号',
  `birthday` date NULL DEFAULT NULL COMMENT '出生日期',
  `sex` varchar(10) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL DEFAULT
NULL COMMENT '性别',
  `phone` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
DEFAULT NULL COMMENT '联系电话',
  `address` varchar(200) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
DEFAULT NULL COMMENT '住址',
  `medical_history` text CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
COMMENT '病史',
  `allergies` text CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL COMMENT
'过敏史',
  `create_time` datetime NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` datetime NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE INDEX `uk_patient_no`(`patient_no` ASC) USING BTREE,
  INDEX `idx_user_id`(`user_id` ASC) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 1 CHARACTER SET = utf8mb4 COLLATE =
utf8mb4_0900_ai_ci COMMENT = '患者表'
```

Patient类：对应患者表，包含患者基本信息属性
【springboot/src/main/java/org/example/springboot/entity/Patient.java】

```java
@Data
@TableName("patient")
@Schema(description = "患者实体类")
public class Patient {
    @TableId(type = IdType.AUTO)
    @Schema(description = "患者ID")
    private Long id;

    @Schema(description = "用户ID")
    private Long userId;

    @Schema(description = "患者编号")
    private String patientNo;
```

半夜撕代码版权所有

```java
    @Schema(description = "姓名")
    private String name;

    @Schema(description = "身份证号")
    private String idCard;

    @Schema(description = "出生日期")
    private LocalDate birthday;

    @Schema(description = "性别")
    private String sex;

    @Schema(description = "联系电话")
    private String phone;

    @Schema(description = "住址")
    private String address;

    @Schema(description = "病史")
    private String medicalHistory;

    @Schema(description = "过敏史")
    private String allergies;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    @Schema(description = "用户信息")
    private User user;
}
```

1.1.2 患者注册DTO

PatientRegisterDTO类：用于患者注册时传递数据
【springboot/src/main/java/org/example/springboot/DTO/PatientRegisterDTO.java】

```java
@Data
@Schema(description = "患者注册DTO")
public class PatientRegisterDTO {
    @Schema(description = "用户信息")
    private UserDTO user;

    @Schema(description = "患者信息")
    private PatientDTO patient;

    @Schema(description = "确认密码")
    @NotBlank(message = "确认密码不能为空")
    private String confirmPassword;

    // 内部类：用户信息DTO
```

半夜撕代码版权所有

```
    @Data
    @Schema(description = "用户信息DTO")
    public static class UserDTO {
        // 用户基本信息字段
    }

    // 内部类：患者信息DTO
    @Data
    @Schema(description = "患者信息DTO")
    public static class PatientDTO {
        // 患者基本信息字段
    }
}
```

1.2 核心属性说明

1. **患者编号(patientNo)**: 系统自动生成的唯一标识，格式为"P"加上UUID的前8位，用于在系统中唯一标识患者
2. **用户ID(userId)**: 关联到用户表的外键，表示患者对应的系统用户账号，可以为空（表示患者未注册系统账号）
3. **身份证号(idCard)**: 患者的身份证号码，用于身份验证和查询
4. **病史(medicalHistory)**: 记录患者的既往病史信息，为文本类型，可以详细描述
5. **过敏史(allergies)**: 记录患者的药物或其他过敏史，为文本类型

二、各功能详细讲解

2.1 患者注册功能

2.1.1 功能概述

患者注册功能允许新用户填写个人信息和健康信息，同时创建系统账号和患者档案。注册过程包括账号信息、基本信息、联系方式和健康信息（选填）的填写，系统会验证各项信息的格式和完整性，确保数据有效。

2.1.2 详细讲解功能实现流程

1. **前端实现**:
   - 用户在Register.vue页面填写注册表单，包括用户名、密码、个人信息和健康信息
   - 表单提交时，前端进行表单验证，确保必填字段已填写且格式正确
   - 验证通过后，调用request.post("/patient/register", registerData)向后端发送注册请求
2. **后端实现**:
   - PatientController的registerPatient方法接收注册请求，验证两次密码是否一致
   - 创建User和Patient对象，并从DTO中复制相关信息
   - 调用PatientService的createPatientWithUser方法同时创建用户和患者记录
   - 服务层进行业务逻辑处理，包括密码加密、生成患者编号等
   - 注册成功后返回患者信息

半夜撕代码版权所有

2.1.3 关键代码讲解

1. **前端注册表单提交**
【vue3/src/views/auth/Register.vue】

```
const handleRegister = () => {
  registerFormRef.value.validate(async valid => {
    if (valid) {
      loading.value = true
      try {
        // 同步用户名和真实姓名
        registerForm.user.name = registerForm.patient.name
        registerForm.user.sex = registerForm.patient.sex
        registerForm.user.phone = registerForm.patient.phone

        // 移除确认密码和协议同意，不需要传递给后端
        const { confirmPassword, agreement, ...registerData } = registerForm

        // 使用患者注册接口
        await request.post("/patient/register", registerData, {
          successMsg: "注册成功，欢迎加入康泰医疗",
          showDefaultMsg: true,
          onSuccess: () => {
            ElMessage({
              type: 'success',
              message: '您的账号已创建成功，正在跳转到登录页面...'
            })
            setTimeout(() => {
              router.push('/login')
            }, 2000)
          }
        })
      } finally {
        loading.value = false
      }
    }
  })
}
```

这段代码实现了表单验证、数据处理和提交注册请求的逻辑。首先验证表单数据的有效性，然后同步用户和患者的共同信息，最后发送POST请求到后端注册接口，成功后跳转到登录页面。

2. **后端注册接口**
【springboot/src/main/java/org/example/springboot/controller/PatientController.java】

```
@Operation(summary = "患者注册")
@PostMapping("/register")
public Result<?> registerPatient(@RequestBody PatientRegisterDTO registerDTO)
{
    LOGGER.info("患者注册: {}", registerDTO);

    // 验证两次密码是否一致
    if
(!registerDTO.getUser().getPassword().equals(registerDTO.getConfirmPassword()
)) {
```

半夜撕代码版权所有

```
        return Result.error("两次输入的密码不一致");
    }

    // 创建用户和患者对象
    User user = new User();
    Patient patient = new Patient();

    // 复制用户信息
    user.setUsername(registerDTO.getUser().getUsername());
    user.setPassword(registerDTO.getUser().getPassword());
    user.setEmail(registerDTO.getUser().getEmail());
    user.setRoleCode(registerDTO.getUser().getRoleCode());

    // 复制患者信息
    patient.setName(registerDTO.getPatient().getName());
    patient.setSex(registerDTO.getPatient().getSex());
    patient.setBirthday(registerDTO.getPatient().getBirthday());
    patient.setIdCard(registerDTO.getPatient().getIdCard());
    patient.setPhone(registerDTO.getPatient().getPhone());
    patient.setAddress(registerDTO.getPatient().getAddress());
    patient.setMedicalHistory(registerDTO.getPatient().getMedicalHistory());
    patient.setAllergies(registerDTO.getPatient().getAllergies());

    // 调用服务创建用户和患者
    Patient newPatient = patientService.createPatientWithUser(user, patient);
    return Result.success(newPatient);
}
```

这段代码处理患者注册请求，首先验证两次密码是否一致，然后将DTO中的数据复制到User和Patient对象中，最后调用服务层方法同时创建用户和患者记录。

3. **服务层实现**
【springboot/src/main/java/org/example/springboot/service/PatientService.java】

```
@Transactional
public Patient createPatientWithUser(User user, Patient patient) {
    // 设置用户角色为患者
    if (user.getRoleCode() == null) {
        user.setRoleCode("PATIENT");
    }

    // 确保用户名和密码存在
    if (StringUtils.isBlank(user.getUsername()) ||
StringUtils.isBlank(user.getPassword())) {
        throw new ServiceException("用户名和密码不能为空");
    }

    // 密码加密
    user.setPassword(userService.encryptPassword(user.getPassword()));

    // 设置默认状态
    user.setStatus(1); // 启用状态

    // 设置默认头像
    if (StringUtils.isBlank(user.getAvatar())) {
        user.setAvatar("/img/default_avatar.png");
```

```
    }

    // 同步用户与患者的共同信息
    user.setName(patient.getName());
    user.setPhone(patient.getPhone());
    user.setSex(patient.getSex());

    // 先创建用户
    userMapper.insert(user);

    // 设置患者关联的用户ID
    patient.setUserId(user.getId());

    // 创建患者
    return createPatient(patient);
}
```

这段代码实现了同时创建用户和患者记录的业务逻辑。首先设置用户角色为患者，然后对密码进行加密处理，设置默认状态和头像，同步用户与患者的共同信息，最后先创建用户记录，再创建患者记录并关联用户ID。

2.1.4 功能实现细节

1. **表单验证**：前端使用Element Plus的表单验证功能，对用户名、密码、邮箱等字段进行格式验证，确保数据符合要求。
2. **密码加密**：后端使用BCryptPasswordEncoder对密码进行加密存储，提高安全性。
3. **患者编号生成**：系统自动生成患者编号，格式为"P"加上UUID的前8位，确保唯一性。
4. **事务管理**：使用@Transactional注解确保用户和患者信息的创建在同一事务中，保证数据一致性。
5. **用户角色设置**：默认将注册用户的角色设置为"PATIENT"，表示患者角色。

2.2 患者管理功能

2.2.1 功能概述

患者管理功能主要面向医院管理人员，提供患者信息的增加、修改、删除和查询功能。管理人员可以通过多种条件搜索患者，查看患者详情，编辑患者信息，以及管理患者与用户的绑定关系。

2.2.2 详细讲解功能实现流程

1. **前端实现**：
   - 在患者管理页面(patient/index.vue)中，管理员可以查看患者列表、搜索患者、新增/编辑/删除患者
   - 页面加载时调用fetchPatients()方法获取患者列表
   - 搜索功能通过调用同一方法并传入搜索条件实现
   - 新增/编辑患者通过弹窗表单实现，提交后调用相应的API
2. **后端实现**：
   - PatientController提供了一系列RESTful接口，包括分页查询、新增、编辑、删除等
   - PatientService实现具体的业务逻辑，如数据验证、关联查询等
   - PatientMapper负责与数据库交互，使用MyBatis-Plus简化CRUD操作

半夜撕代码版权所有

2.2.3 关键代码讲解

1. **前端患者列表获取**
   【vue3/src/views/backend/patient/index.vue】

```
const fetchPatients = async () => {
  loading.value = true
  try {
    await request.get('/patient/page', {
      name: searchForm.name,
      idCard: searchForm.idCard,
      phone: searchForm.phone,
      username: searchForm.username,
      currentPage: currentPage.value,
      size: pageSize.value
    }, {
      onSuccess: (res) => {
        tableData.value = res.records
        total.value = res.total
      }
    })
  } catch (error) {
    console.error('获取患者列表失败:', error)
  } finally {
    loading.value = false
  }
}
```

   这段代码实现了分页获取患者列表的功能，通过request工具发送GET请求到'/patient/page'接口，传入搜索条件和分页参数，获取患者列表数据并更新到表格中。

2. **后端分页查询实现**
   【springboot/src/main/java/org/example/springboot/service/PatientService.java】

```
public Page<Patient> getPatientsByPage(String name, String idCard, String
phone, String username, Integer currentPage, Integer size) {
    Page<Patient> page = new Page<>(currentPage, size);

    // 如果需要按关联用户名查询，使用自定义SQL查询
    if (StringUtils.isNotBlank(username)) {
        return patientMapper.selectPatientsByUsername(page, name, idCard,
phone, username);
    }

    // 常规查询条件
    LambdaQueryWrapper<Patient> queryWrapper = new LambdaQueryWrapper<>();

    // 添加查询条件
    if (StringUtils.isNotBlank(name)) {
        queryWrapper.like(Patient::getName, name);
    }
    if (StringUtils.isNotBlank(idCard)) {
        queryWrapper.like(Patient::getIdCard, idCard);
    }
    if (StringUtils.isNotBlank(phone)) {
```

半夜撕代码版权所有

```
        queryWrapper.like(Patient::getPhone, phone);
    }

    // 按创建时间降序排序
    queryWrapper.orderByDesc(Patient::getCreateTime);

    Page<Patient> patientPage = patientMapper.selectPage(page, queryWrapper);

    // 查询关联的用户信息
    for (Patient patient : patientPage.getRecords()) {
        if (patient.getUserId() != null) {
            User user = userMapper.selectById(patient.getUserId());
            patient.setUser(user);
        }
    }

    return patientPage;
}
```

这段代码实现了分页查询患者列表的业务逻辑。首先判断是否需要按关联用户名查询，如果需要则调用自定义SQL查询方法；否则使用MyBatis-Plus的LambdaQueryWrapper构建查询条件，包括姓名、身份证号和手机号的模糊匹配，并按创建时间降序排序。查询结果中，还会加载每个患者关联的用户信息。

3. **患者删除功能**

【springboot/src/main/java/org/example/springboot/service/PatientService.java】

```
@Transactional
public void deletePatient(Long id) {
    // 检查患者是否存在
    Patient patient = patientMapper.selectById(id);
    if (patient == null) {
        throw new ServiceException("患者不存在");
    }

    // 检查患者是否有关联预约
    LambdaQueryWrapper<Appointment> appointmentQueryWrapper = new
LambdaQueryWrapper<>();
    appointmentQueryWrapper.eq(Appointment::getPatientId, id);
    long appointmentCount =
appointmentMapper.selectCount(appointmentQueryWrapper);
    if (appointmentCount > 0) {
        throw new ServiceException("该患者已有预约记录，不能删除");
    }

    // 检查患者是否有关联就诊记录
    LambdaQueryWrapper<MedicalRecord> medicalRecordQueryWrapper = new
LambdaQueryWrapper<>();
    medicalRecordQueryWrapper.eq(MedicalRecord::getPatientId, id);
    long medicalRecordCount =
medicalRecordMapper.selectCount(medicalRecordQueryWrapper);
    if (medicalRecordCount > 0) {
        throw new ServiceException("该患者已有就诊记录，不能删除");
    }
```

```
    // 检查患者是否有关联处方
    LambdaQueryWrapper<Prescription> prescriptionQueryWrapper = new
LambdaQueryWrapper<>();
    prescriptionQueryWrapper.eq(Prescription::getPatientId, id);
    long prescriptionCount =
prescriptionMapper.selectCount(prescriptionQueryWrapper);
    if (prescriptionCount > 0) {
        throw new ServiceException("该患者已有处方记录，不能删除");
    }

    if (patientMapper.deleteById(id) <= 0) {
        throw new ServiceException("患者删除失败");
    }
}
```

这段代码实现了删除患者的业务逻辑。首先检查患者是否存在，然后检查患者是否有关联的预约记录、就诊记录和处方记录，如果有则不允许删除，确保数据的完整性和一致性。最后执行删除操作，并处理删除失败的情况。

2.2.4 功能实现细节

1. **多条件搜索**：支持按姓名、身份证号、手机号和关联用户名进行模糊搜索。
2. **关联查询**：查询患者信息时，同时加载关联的用户信息，提供完整的数据视图。
3. **数据验证**：删除患者前，检查是否有关联的预约、就诊记录和处方，避免数据不一致。
4. **批量操作**：支持批量删除患者，提高操作效率。
5. **用户绑定管理**：提供绑定和解绑患者与用户关系的功能，方便管理患者账号。

2.3 患者个人中心功能

2.3.1 功能概述

患者个人中心功能允许已登录的患者用户查看和管理自己的个人信息、患者信息和账号安全设置。用户可以更新个人资料、修改密码，以及添加或编辑患者健康信息，实现自助管理个人健康档案。

2.3.2 详细讲解功能实现流程

1. **前端实现**：
   - 在个人中心页面(profile/index.vue)中，患者可以查看和编辑个人信息、患者信息和修改密码
   - 页面加载时调用getUserInfo()和fetchPatientInfo()方法获取用户和患者信息
   - 通过表单提交更新个人信息、患者信息或修改密码
2. **后端实现**：
   - PatientController提供了获取当前患者信息、更新患者信息等接口
   - PatientService实现相关业务逻辑，包括获取当前登录用户关联的患者信息
   - 使用JwtTokenUtils工具类获取当前登录用户信息

半夜撕代码版权所有

2.3.3 关键代码讲解

1. **获取当前患者信息**
   【vue3/src/views/profile/index.vue】

```
const fetchPatientInfo = async () => {
  if (!userStore.isLoggedIn) {
    ElMessage.warning('请先登录');
    return;
  }

  patientLoading.value = true;
  try {
    await request.get('/patient/current', {}, {
      showDefaultMsg: false,
      onSuccess: (res) => {
        patientInfo.value = res;
      },
      onError: (err) => {
        // 如果是未找到患者信息的错误，不显示错误提示
        if (err && err.message && err.message.includes('未找到患者信息')) {
          patientInfo.value = null;
        }
      }
    });
  } catch (error) {
    console.error('获取患者信息失败:', error);
  } finally {
    patientLoading.value = false;
  }
};
```

   这段代码实现了获取当前登录用户的患者信息功能。首先检查用户是否已登录，然后发送GET请求到'/patient/current'接口获取患者信息。如果获取成功，则更新patientInfo变量；如果是因为未找到患者信息而失败，则将patientInfo设为null，表示用户未关联患者信息。

2. **后端获取当前患者信息**
   【springboot/src/main/java/org/example/springboot/service/PatientService.java】

```
public Patient getCurrentPatient() {
    User currentUser = JwtTokenUtils.getCurrentUser();
    if (currentUser == null) {
        throw new ServiceException("用户未登录");
    }

    return getPatientByUserId(currentUser.getId());
}

public Patient getPatientByUserId(Long userId) {
    Patient patient = patientMapper.selectOne(
        new LambdaQueryWrapper<Patient>()
            .eq(Patient::getUserId, userId)
    );

    if (patient == null) {
```

半夜撕代码版权所有

```
        throw new ServiceException("未找到患者信息");
    }

    // 查询关联的用户信息
    User user = userMapper.selectById(userId);
    patient.setUser(user);

    return patient;
}
```

这段代码实现了获取当前登录用户关联的患者信息功能。首先通过JwtTokenUtils工具类获取当前登录用户，然后调用getPatientByUserId方法根据用户ID查询患者信息。如果找不到关联的患者信息，则抛出异常；否则加载关联的用户信息并返回患者对象。

3. **添加/编辑患者信息**
【vue3/src/views/profile/index.vue】

```
const submitPatientForm = () => {
  patientFormRef.value.validate(async (valid) => {
    if (valid) {
      // 确保userId是当前用户ID
      patientForm.userId = userStore.userInfo?.id;

      if (patientDialogType.value === 'add') {
        // 新增患者
        await request.post('/patient', patientForm, {
          successMsg: '添加患者信息成功',
          onSuccess: () => {
            patientDialogVisible.value = false;
            fetchPatientInfo();
          }
        });
      } else {
        // 编辑患者
        await request.put(`/patient/${patientForm.id}`, patientForm, {
          successMsg: '更新患者信息成功',
          onSuccess: () => {
            patientDialogVisible.value = false;
            fetchPatientInfo();
          }
        });
      }
    }
  });
};
```

这段代码实现了提交患者表单的功能，包括添加和编辑两种情况。首先验证表单数据的有效性，然后确保患者关联的用户ID是当前登录用户的ID。根据操作类型（add或edit）选择发送POST请求创建新患者，或PUT请求更新现有患者信息。操作成功后，关闭对话框并重新获取患者信息。

半夜撕代码版权所有

2.3.4 功能实现细节

1. **用户关联**：患者信息与用户账号关联，确保用户只能查看和编辑自己的患者信息。
2. **表单验证**：使用Element Plus的表单验证功能，确保输入的患者信息格式正确。
3. **条件显示**：根据是否已有患者信息，显示不同的界面内容（添加按钮或患者详情）。
4. **信息同步**：用户更新个人信息时，相关字段会同步更新到患者信息中。
5. **安全控制**：后端通过JWT令牌获取当前登录用户，确保用户只能操作自己的数据。

2.4 患者与用户绑定功能

2.4.1 功能概述

患者与用户绑定功能允许管理员将患者信息与系统用户账号进行关联或解除关联。这个功能主要用于管理已有患者信息但尚未注册系统账号的情况，或者需要更换关联账号的情况。

2.4.2 详细讲解功能实现流程

1. **前端实现**：
   - 在患者管理页面中，编辑患者信息时可以选择关联的用户账号
   - 通过下拉菜单选择现有用户，或者创建新用户进行关联
2. **后端实现**：
   - PatientController提供了绑定和解绑患者与用户关系的接口
   - PatientService实现具体的业务逻辑，包括检查用户是否已绑定其他患者等

2.4.3 关键代码讲解

1. **绑定用户与患者关系**
   【springboot/src/main/java/org/example/springboot/service/PatientService.java】

```
@Transactional
public void bindUserToPatient(Long patientId, Long userId) {
    // 检查患者是否存在
    Patient patient = patientMapper.selectById(patientId);
    if (patient == null) {
        throw new ServiceException("患者不存在");
    }

    // 检查用户是否存在
    User user = userMapper.selectById(userId);
    if (user == null) {
        throw new ServiceException("用户不存在");
继续患者模块讲解文档的内容：
```

```
    // 检查用户是否已绑定其他患者
    Patient existPatient = patientMapper.selectOne(
        new LambdaQueryWrapper<Patient>()
            .eq(Patient::getUserId, userId)
    );
    if (existPatient != null && !existPatient.getId().equals(patientId)) {
        throw new ServiceException("该用户已绑定其他患者");
    }
```

半夜撕代码版权所有

```
        // 更新患者关联的用户ID
        patient.setUserId(userId);
        patient.setUpdateTime(LocalDateTime.now());

        if (patientMapper.updateById(patient) <= 0) {
            throw new ServiceException("绑定用户失败");
        }
    }
```

这段代码实现了绑定患者与用户关系的业务逻辑。首先检查患者和用户是否存在，然后检查用户是否已经绑定了其他患者，如果是则不允许绑定。最后更新患者记录，设置关联的用户ID和更新时间。

2. **解绑用户与患者关系**
【springboot/src/main/java/org/example/springboot/service/PatientService.java】

```
public void unbindUserFromPatient(Long patientId) {
    // 检查患者是否存在
    Patient patient = patientMapper.selectById(patientId);
    if (patient == null) {
        throw new ServiceException("患者不存在");
    }

    // 解除绑定
    patient.setUserId(null);
    patient.setUpdateTime(LocalDateTime.now());

    if (patientMapper.updateById(patient) <= 0) {
        throw new ServiceException("解绑用户失败");
    }
}
```

这段代码实现了解绑患者与用户关系的业务逻辑。首先检查患者是否存在，然后将患者的userId字段设为null，并更新更新时间。最后保存更改，完成解绑操作。

3. **前端绑定用户实现**
【vue3/src/views/backend/patient/index.vue】

```
// 提交表单
const submitForm = async () => {
  // ... 表单验证代码 ...

  try {
    loading.value = true;

    if (dialogType.value === 'edit') {
      const { data: currentPatient } = await
request.get(`/patient/${patientForm.id}`, {}, {
        showDefaultMsg: false
      });

      if (currentPatient && currentPatient.userId !== patientForm.userId) {
        if (currentPatient.userId) {
          await request.put(`/patient/unbind/${patientForm.id}`, {}, {
            showDefaultMsg: false,
```

```
            onError: (error) => {
              ElMessage.error('解绑原关联用户失败: ' + (error.message || '未知错误'));
              throw new Error('解绑用户失败');
            }
          });
        }

        if (patientForm.userId) {
          await request.put(`/patient/bind/${patientForm.id}/${patientForm.userId}`, {}, {
            showDefaultMsg: false,
            onError: (error) => {
              ElMessage.error('绑定新关联用户失败: ' + (error.message || '未知错误'));
              throw new Error('绑定用户失败');
            }
          });
        }
      }

      // ... 更新患者其他信息的代码 ...
    }
  } catch (error) {
    console.error('操作失败:', error);
  } finally {
    loading.value = false;
  }
}
```

这段代码实现了前端编辑患者信息时处理用户绑定关系的逻辑。首先获取当前患者信息，然后比较表单中的userId和当前患者的userId是否不同。如果不同，则先解绑原关联用户（如果有），再绑定新用户（如果有）。整个过程使用try-catch块处理可能的错误。

2.4.4 功能实现细节

1. **唯一绑定**：一个用户只能绑定一个患者，避免数据混乱。
2. **事务管理**：使用@Transactional注解确保绑定操作的原子性。
3. **错误处理**：详细的错误提示，帮助用户理解操作失败的原因。
4. **权限控制**：只有管理员可以执行绑定和解绑操作，普通用户无权限。
5. **数据一致性**：绑定和解绑操作会更新患者的更新时间，保持数据的时间一致性。

三、总结

3.1 患者注册功能实现

患者注册功能通过前端Register.vue组件实现，用户填写包含账号信息、基本信息、联系方式和健康信息的表单。表单数据经过前端验证后，通过request.post("/patient/register", registerData)提交到后端。后端PatientController的registerPatient方法接收请求，验证密码一致性，然后创建User和Patient对象。PatientService的createPatientWithUser方法处理核心业务逻辑，包括密码加密、设置默认值、生成患者编号等。注册成功后，前端会显示成功提示并跳转到登录页面。整个过程确保了数据的有效性和安全性，为患者提供了便捷的注册入口。

半夜撕代码版权所有

3.2 患者管理功能实现

患者管理功能主要面向医院管理人员，通过前端patient/index.vue组件实现。页面加载时调用fetchPatients()方法获取患者列表，支持按姓名、身份证号、手机号和关联用户名进行多条件搜索。后端PatientController提供了分页查询接口，PatientService的getPatientsByPage方法使用MyBatis-Plus构建查询条件，实现灵活的查询功能。管理员可以通过界面添加、编辑和删除患者信息，系统会进行必要的数据验证，如检查患者是否有关联的预约、就诊记录和处方，确保数据的完整性和一致性。

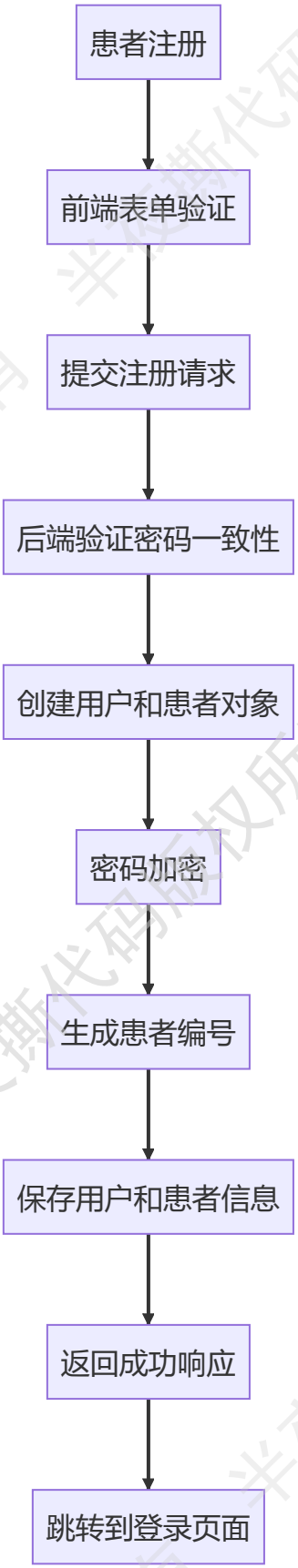
3.3 患者个人中心功能实现

患者个人中心功能通过前端profile/index.vue组件实现，为患者用户提供自助管理个人信息的界面。页面包含基本信息、患者信息和密码修改三个标签页。用户可以更新个人资料、上传头像、修改密码，以及添加或编辑患者健康信息。后端PatientController和PatientService提供了获取当前患者信息和更新患者信息的接口，通过JwtTokenUtils工具类获取当前登录用户，确保用户只能操作自己的数据。如果用户尚未添加患者信息，系统会显示添加按钮；否则显示详细的患者信息，包括病史和过敏史等健康数据。

3.4 患者与用户绑定功能实现

患者与用户绑定功能允许管理员将患者信息与系统用户账号进行关联或解除关联。PatientController提供了bindUserToPatient和unbindUserFromPatient接口，PatientService实现了相应的业务逻辑。绑定操作会检查用户是否已绑定其他患者，确保一个用户只能绑定一个患者。解绑操作会清除患者的userId字段，断开与用户的关联。前端在编辑患者信息时，可以通过下拉菜单选择关联的用户，或者创建新用户进行关联。整个功能设计确保了数据的一致性和操作的安全性。

半夜撕代码版权所有


患者注册
前端表单验证
提交注册请求
后端验证密码一致性
创建用户和患者对象
密码加密
生成患者编号
保存用户和患者信息
返回成功响应
跳转到登录页面

半夜撕代码版权所有